

Field Expense PWA

Solo Developer Roadmap — Strategic Guide & Phased Build Plan

This document covers the full strategic picture for building a field-first expense capture PWA targeting the Egyptian and MENA market: market research, competitor gaps, AI advantages, and a phased development plan for a solo developer.

1. Market Research — Competitor Landscape

Competitor	Position	Key Gaps for Egypt
Expensify	Global leader	No EGP native pricing, no Egyptian Arabic voice, no ETA QR, no local ERP integration
SAP Concur	Enterprise standard	Requires internet, clunky UI, expensive, Arabic is a translation layer not native UX
Zoho Expense	MENA presence	Tuned for UAE/Saudi VAT law (ZATCA), not Egyptian ETA. Voice is English-only
Daftra / Edara	Egyptian market	Accounting backends only — no mobile capture layer. Field worker still uses WhatsApp
WhatsApp + Spreadsheet	Actual current workflow	This is your real competitor — your UX must beat sending a photo message

2. Market Gaps You Can Fill

1. Egyptian Arabic Voice Input

Every competitor uses English-only voice. Egyptian Arabic dialect is phonetically distinct. A field worker in Assiut speaks differently from what Whisper defaults to without prompting.

2. ETA-Aware Receipt Parsing

Egypt's e-invoicing mandate encodes structured data in a QR on every receipt. No competitor parses it. Your app can auto-fill from government-verified invoice data.

3. The WhatsApp Gap

Right now field workers send receipt photos to WhatsApp groups. This is the workflow you're replacing — not Expensify. Your UX must be as fast as WhatsApp.

4. Project-Based Attribution

In event management, construction, and contracting — expenses must map to a project or work order. Global tools don't handle Egyptian project-centric accounting natively.

3. AI Competitive Advantages

• **Voice-First, Form-Free Capture**

GPT-4o-mini-transcribe processes Egyptian Arabic voice and extracts structured fields. The user never sees a form — they speak naturally and see fields pre-filled.

• **Self-Learning Correction Loop**

Accountant corrections feed back as few-shot examples into future extraction prompts. After 200 corrections per company, extraction accuracy compounds dramatically.

• **ETA QR Extraction**

Detects and decodes Egyptian e-invoice QR codes from receipt photos. Extracts tax registration number, amount, and date automatically — zero manual entry.

• **Anomaly Detection**

Perceptual hashing catches duplicate receipts. Statistical analysis flags outliers. Velocity checks catch suspicious rapid submissions. All advisory, never auto-rejecting.

• **Vendor Intelligence**

Caches vendor names by tax registration number after first encounter. Over time builds a company-specific vendor database that accelerates future submissions.

4. Phased Development Plan

Phase 1: Foundation

Weeks 1–3

Design System

FOUNDATION

Build this

- ✓ Arabic-first CSS with RTL logical properties
- ✓ Font: IBM Plex Arabic + English pairing
- ✓ CSS variables: colors, spacing, radius, shadows
- ✓ Core components: Button, Input, Card, Badge, Toast
- ✓ Dark/light mode from day one

Defer / Skip

- Complex animations — add last
- Breakpoints above tablet — mobile-only in Phase 1
- Multiple icon libraries — pick one (Tabler)

Why this order: Every screen reuses these tokens. One week here saves three weeks of inconsistency later.

Project Scaffold

FOUNDATION

Build this

- ✓ Vite + React 18 + TypeScript (strict mode)
- ✓ vite-plugin-pwa: manifest, icons, service worker
- ✓ Dexie.js IndexedDB schema mirroring server schema
- ✓ React Router v6: /field/* and /accountant/* groups
- ✓ FastAPI + Docker Compose: Postgres + API + Nginx

Defer / Skip

- State management libs (Redux/Zustand) — useState+Context is enough
- Testing setup — add in Phase 3 when API stabilizes
- CI/CD pipeline — deploy manually until Phase 3

Why this order: Get the skeleton right. Two days of scaffold prevents two weeks of restructuring pain.

Auth + Users

FOUNDATION

Build this

- ✓ JWT-based auth: FastAPI + python-jose
- ✓ Three roles: field_worker, accountant, admin
- ✓ Route guards in React: useAuth hook
- ✓ Company/tenant model — data isolated per company

Defer / Skip

- SSO / OAuth (Google, Microsoft) — Phase 4+
- Fine-grained permissions — three roles is enough for v1
- Two-factor auth — not needed for first deployment

Why this order: Auth touches everything. Simple, clean implementation done once — stop thinking about it.

Data Schema

FOUNDATION

Build this

- ✓ expenses: id, user_id, project_id, amount, currency, category, vendor, receipt_url, status
- ✓ projects: id, company_id, name, budget, active
- ✓ correction_feedback: expense_id, field, ai_value, corrected_value
- ✓ companies + users with company_id FK for multi-tenancy
- ✓ Alembic for migrations — never hand-edit the DB

Defer / Skip

- Complex chart of accounts mapping — Phase 4
- Analytics tables — derive from expenses on query
- Attachments table — receipt URL in expense row, images in S3/R2

Why this order: Design schema before any API code. Changing mid-development is extremely painful.

Phase 2: Core Capture

Weeks 4–7

Voice Capture — Main Differentiator

CORE FEATURE

Build this

- ✓ MediaRecorder API in browser — webm/opus, max 60s
- ✓ FastAPI endpoint → Whisper (gpt-4o-mini-transcribe)
- ✓ System prompt: Egyptian Arabic, extract JSON {amount, currency, category, vendor, project_hint}
- ✓ Few-shot examples injected from correction_feedback table
- ✓ Return pre-filled form in <2 seconds
- ✓ Fallback: low-confidence fields left blank and highlighted

Defer / Skip

- On-device speech recognition — Whisper far more accurate for Egyptian Arabic
- Custom model fine-tuning — prompt engineering gets 90% of the way
- Real-time streaming transcription — batch is fine

Why this order: No one does Egyptian Arabic voice-to-expense. Spend real time on the extraction prompt — 80% of value is here.

Receipt OCR Pipeline

CORE FEATURE

Build this

- ✓ — native camera
- ✓ Client-side compression: max 1200px, JPEG 0.85 before upload
- ✓ GPT-4o vision: extract amount, vendor, date, category from Arabic/English receipt
- ✓ ETA QR detection: pyzbar decodes QR → extract tax reg number, amount, date
- ✓ Merge voice + image if both provided

Defer / Skip

- Custom OCR model — GPT-4o vision outperforms anything you'd train
- PDF invoice support — Phase 4, photos first
- Batch scanning — single receipt per expense for v1

Why this order: ETA QR integration is your killer feature. No competitor in Egypt does this. Auto-fills from government-verified data.

Expense Form — Critical UX

CORE FEATURE

Build this

- ✓ Pre-filled from AI — user should rarely type anything
- ✓ Fields: Amount (large, prominent), Category, Vendor (autocomplete), Project, Notes
- ✓ Single large Submit button — one tap to accountant queue
- ✓ Auto-save draft to IndexedDB every 5 seconds
- ✓ Arabic labels default, English toggle in settings

Defer / Skip

- Multi-expense reports — one at a time for v1
- Mileage / per-diem — Phase 5
- Multi-currency — EGP only in v1

Why this order: Target: under 15 seconds from open to submit. Test with a real field worker, not a developer, before shipping.

Offline + Background Sync

CORE FEATURE

Build this

- ✓ Dexie.js: save locally first, server second — always
- ✓ Workbox service worker: cache app shell, network-first API
- ✓ Background Sync API: replay queue on reconnect
- ✓ Status indicator: 'Saved locally — syncing when online'

Defer / Skip

- Complex merge conflict resolution — last-write-wins for v1
- Offline accountant dashboard — accountants have connectivity
- Push-based real-time updates — polling every 30s is fine

Why this order: Egypt has patchy connectivity outside Cairo. Offline-first is not optional — it's the baseline.

Phase 3: Review Desk

Weeks 8–10

Accountant Review Queue

CORE FEATURE

Build this

- ✓ Pending list: sort by date, amount, project, employee
- ✓ Expense detail: receipt image (zoomable), extracted fields, confidence indicators
- ✓ Actions: Approve, Reject with reason, Edit fields
- ✓ Bulk approve for obvious/repeated expenses
- ✓ Filter by project, date range, employee, status

Defer / Skip

- GL codes, cost centers — Phase 4 with accounting integration
- Export to Excel — Phase 4
- Audit log — Phase 4

Why this order: Accountant is your second customer. Painful review = back to WhatsApp. Make the queue fast and scannable.

AI Correction Feedback Loop

AI MOAT

Build this

- ✓ On correction: INSERT correction_feedback(company_id, field, ai_guess, human_answer)
- ✓ Before extraction: SELECT last 10 relevant examples WHERE company_id = \$1
- ✓ Inject as few-shot in system prompt: 'For this company: vendor XYZ → Maintenance'
- ✓ Track correction rate per field — surface which fields need attention
- ✓ After 50+ corrections: model effectively learns company vocabulary

Defer / Skip

- Fine-tuning a custom model — few-shot achieves same with zero training cost
- Cross-company learning — corrections stay private per company

Why this order: This is the compounding moat. Every correction makes it smarter. After 3 months, extraction is dramatically better than day one.

Web Push Notifications

CORE FEATURE

Build this

- ✓ web-push library in FastAPI backend
- ✓ Field worker: 'Expense EGP 250 approved' / 'Rejected: missing vendor'
- ✓ Accountant: '3 new expenses pending' — batched, not per-expense
- ✓ Permission prompt after first successful submission

Defer / Skip

- Email notifications — Phase 4
- In-app real-time feed — polling is fine
- SMS fallback

Why this order: Closes the feedback loop. Without it, field workers don't know if their expense was accepted. This is a trust feature.

Phase 4: Integration & Scale

Weeks 11–14

Accounting System Integration

PHASE 4

Build this

- ✓ Abstraction layer: ExpenseExporter interface with push(expense)
- ✓ Adapter 1: Zoho Books — approved expense → Zoho expense API
- ✓ Adapter 2: Odoo — approved expense → journal entry via XML-RPC
- ✓ Adapter 3: Daftra — API or CSV export
- ✓ Category → chart of accounts mapping on setup

Defer / Skip

- SAP/Oracle — not your market (enterprise-only)
- Bi-directional sync — push only

Why this order: Abstraction layer first — adding a second system later is just a new adapter, not a refactor.

ETA QR Code Parsing

AI MOAT

Build this

- ✓ pyzbar/zxing-cpp in FastAPI: detect + decode QR from image
- ✓ ETA QR contains: tax reg number, invoice UUID, amount, date, seller
- ✓ Auto-fill vendor, amount, date — zero typing required
- ✓ 'ETA verified' badge on expense — accountant trusts immediately

Defer / Skip

- ETA API validation — QR decode is enough for v1
- Forgery detection — flag if QR data conflicts with GPT extraction

Why this order: Available nowhere else in Egyptian market. ETA-verified expense takes zero seconds to process.

AI Anomaly Detection

AI MOAT

Build this

- ✓ Duplicate detection: perceptual hash of images, flag >90% similarity
- ✓ Statistical outlier: >2 std deviations from user+category average
- ✓ Velocity check: 3+ expenses in 10 minutes — flag for review
- ✓ Vendor/category mismatch detection
- ✓ Advisory badges on queue — not blocking, never auto-reject

Defer / Skip

- ML-based fraud classification — statistical rules catch 80% with 5% effort
- Automated rejection — always keep human in the loop

Why this order: Turns the app from a data-entry tool into a financial controls system. Justifies enterprise pricing.

Spend Analytics Dashboard

PHASE 4

Build this

- ✓ Bar chart: spend by project (30/90 days)
- ✓ Donut: spend by category
- ✓ Line chart: spend trend per team over time
- ✓ Budget vs actual per project
- ✓ Export to CSV / Excel

Defer / Skip

- Custom date ranges — presets (week/month/quarter) are enough
- Predictive analytics — Phase 5

Why this order: This is what justifies the subscription to the CFO. Both operational users and finance leadership need to love the product.

5. Solo Dev Guiding Principles

Design first, then build

Spend week 1 purely on the design system — RTL, Arabic fonts, component library. Zero backend. This pays off across every phase.

Field UX before accountant UX

The hardest UX is the field worker's. If capture is painful, nothing else matters. Nail that before building the review desk.

Ship Phase 2 to real users

After Phase 2 you have a usable product. Deploy it. Get Pillars field team using it. Real data shapes everything in Phase 3+.

Defer integrations

Don't touch Zoho/Odoo APIs until Phase 4. Build a clean internal data model first. Integrations are adapters — easy to add later.

6. Proposed Tech Stack

Layer	Technology	Rationale
Frontend	React 18 + Vite + TypeScript	Already your stack from Pillars Voice Ops
PWA	vite-plugin-pwa + Workbox	Service worker, offline, background sync
Offline DB	Dexie.js (IndexedDB)	Already proven in your Pillars project
Routing	React Router v6	Two route groups: /field and /accountant
Backend	FastAPI (Python)	Easy OpenAI SDK, Pillow for images, Pydantic
Database	PostgreSQL + Alembic	Migrations, JSON fields for AI metadata
Voice AI	OpenAI Whisper (gpt-4o-mini-transcribe)	Best accuracy for Egyptian Arabic
Receipt AI	GPT-4o Vision	Outperforms any custom OCR for Arabic receipts
QR Decode	pyzbar / zxing-cpp	Decode ETA e-invoice QR from receipt images
Push Notifs	web-push (Python)	Web Push API — no native app needed
Hosting	VPS + Docker Compose	Nginx + FastAPI + Postgres — your existing infra
Storage	Cloudflare R2 / S3	Receipt image storage, URL in expense row
i18n	react-i18next	Arabic RTL default, English toggle

The single most important insight: your competition in Egypt is not Expensify — it's WhatsApp. If your app is harder to use than a phone photo + message, adoption dies. AI makes the difference. The PWA makes it accessible. Egyptian Arabic voice makes it actually usable.